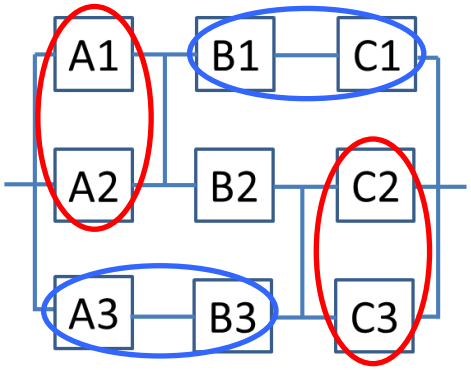
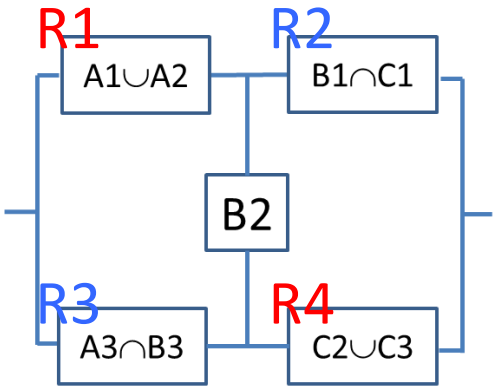
Home Exercise: Reliability

PP scheme



Bridge



R1	R2	B2	R3	R4	Sys	Probability
1	1	1	1	1	1	R1R2
1	1	1	1	0	1	
-	-	-	-	-	-	
1	1	0	0	0	1	
1	0	1	1	1	1	R1R2'B2R3R4
1	0	1	1	0	0	0
1	0	1	0	1	1	R1R2'B2R3'R4
1	0	1	0	0	0	0
-	-	-	-	-	-	
0	0	0	0	0	0	0

$R1 = (1-(1-R_A)^2)$
$R2 = R_B R_C$
$R3 = R_A R_B$
$R4 = (1-(1-R_C)^2)$

$R_{bridge} = R1R2 + R1R2'B2R4 + R1'R2B2R3 + R1'R2B2'R3R4 + R1'R2'R3R4$

Byzantine Fault Tolerance

(Fault Tolerance in Distributed Systems)

2110473 Fault Tolerant Computing

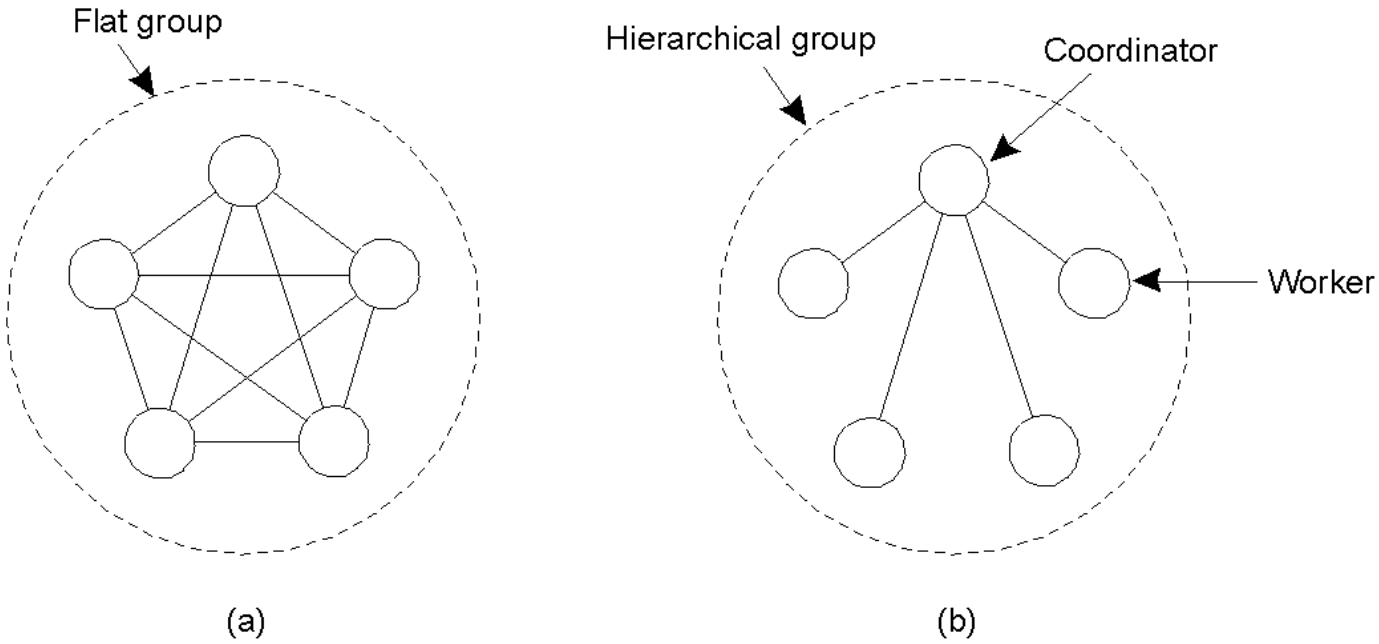
Several sources

Contents

- Process Groups
- Fault Tolerance in Process Groups
- Byzantine Generals Problem
- Oral Message Algorithm
- Blockchain
- Reliable Client-Server Communication

Process Groups

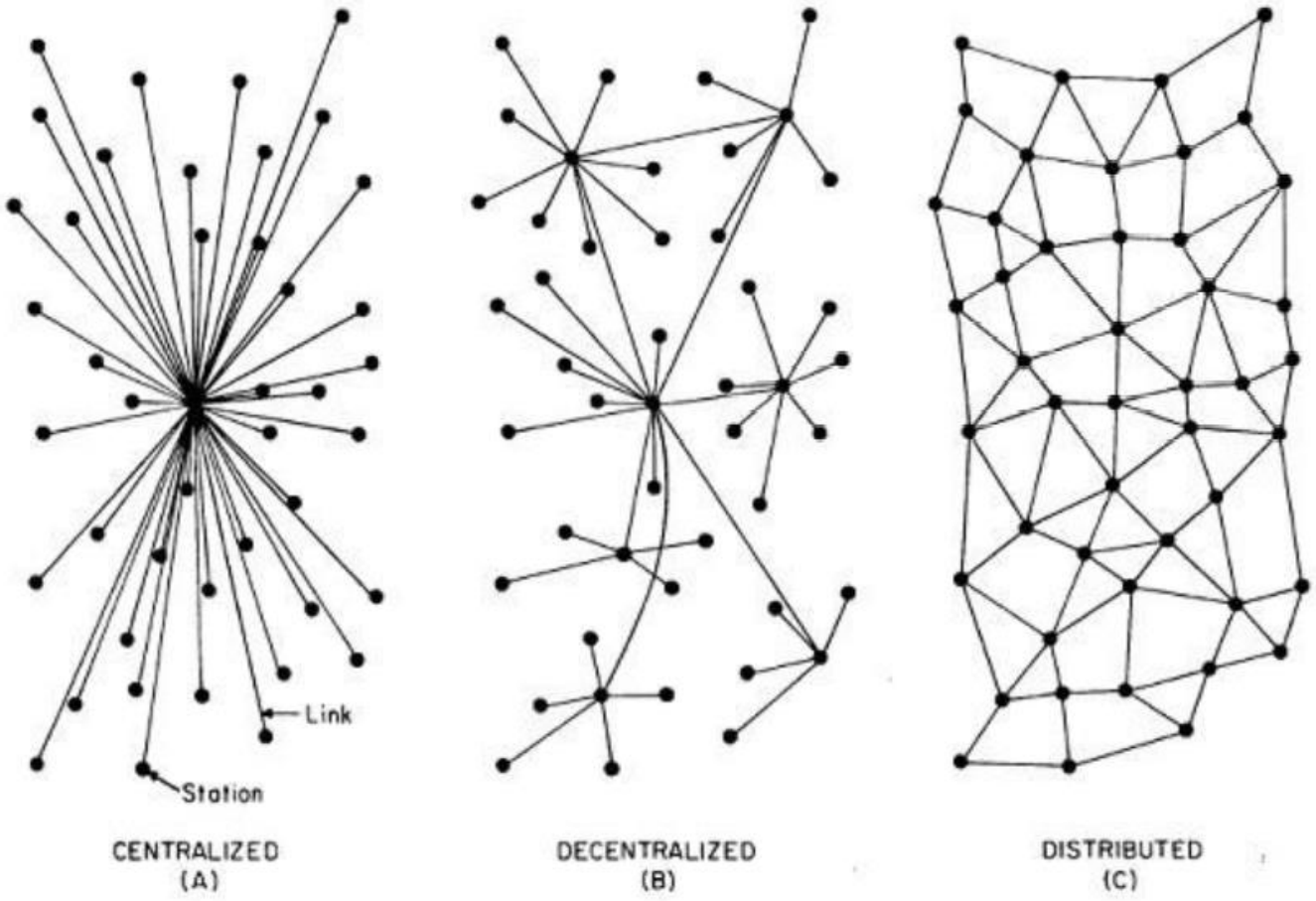
- All members of a group receive a message sent to the group
- If one process fails, others can take over
- Can be dynamic; processes can have multiple memberships



(a) Communication in a **flat group**: *No Single Point-of-Failure, Complicated Decision Making*

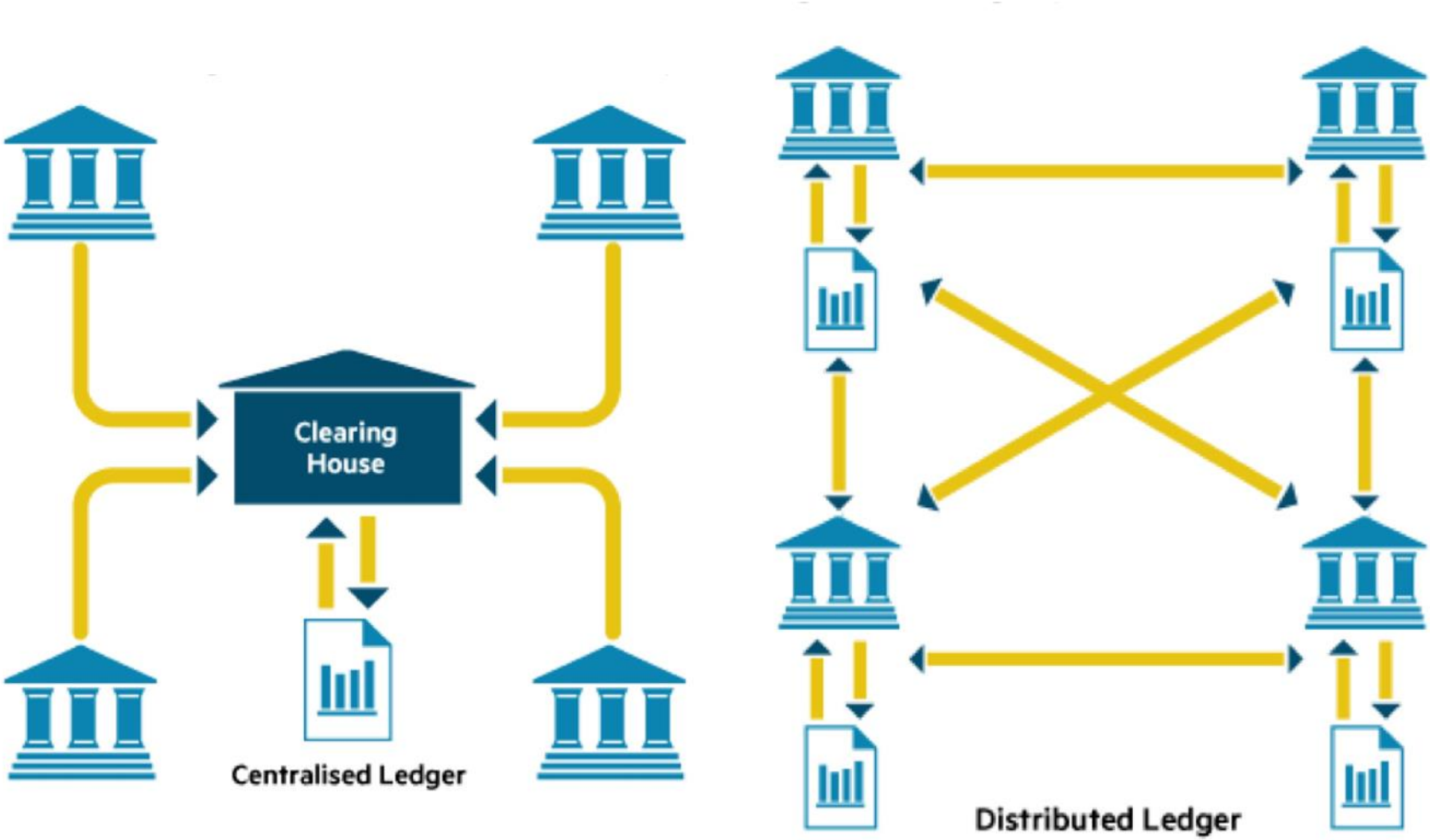
(b) Communication in a **hierarchical group**: *Single Point-of-Failure, Quick Decision Making*

Process Group Architecture



Central Authority → Consensus Protocol (Agreement)

Centralized & Distributed Ledger



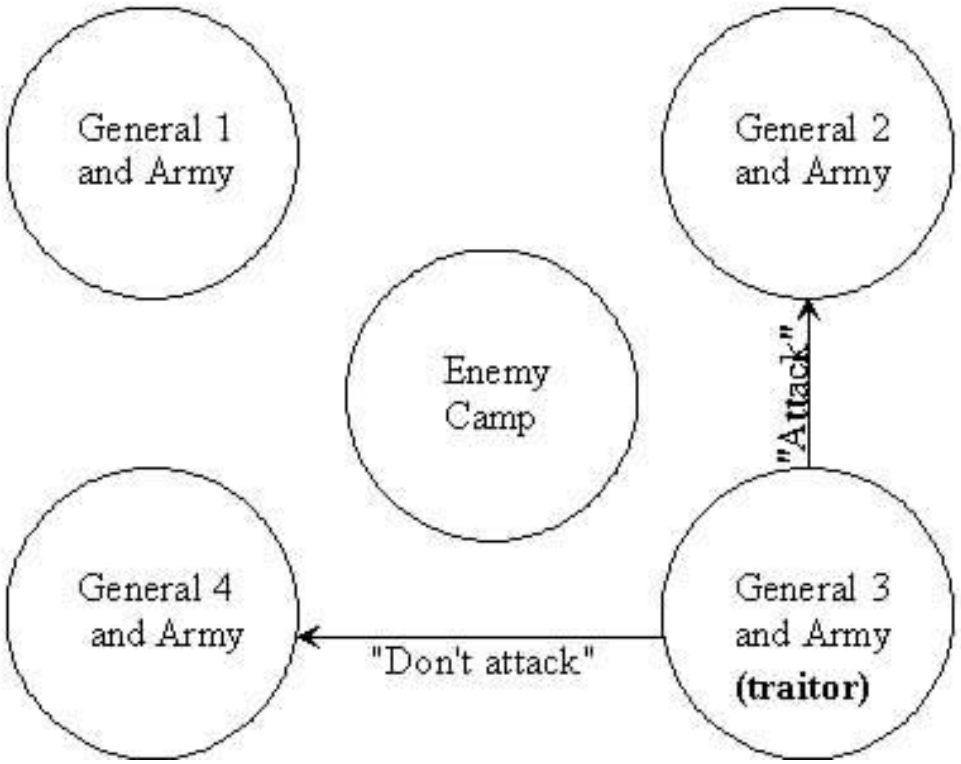
Process Group Resilience

- Organize several identical processes into a group
- When a message is send to a group, all members of the group receives it
- If one process in a group fails (no matter what reason), hopefully some other process can take over for it
- The purpose of introducing groups is to allow processes to deal with collections of processes as a single abstraction.
- Important design issue is how to reach agreement within a process group when one or more of its members cannot be trusted to give correct answers.

Fault Tolerance in Process Groups

- A system is ***k fault tolerant*** if it can survive faults in ***k*** components and still meet its specification
- If failures are safe (silent), then ***k+1*** processes are sufficient to get ***k*** fault tolerance
- In case of arbitrary failures, ***2k+1*** processes are required (since ***k*** failing processes can all generate the same result by chance)
  - This assumes that each process reaches its decision independently
  - What if processes gang up to produce wrong results? General problem is having a process group reach an **agreement**
  - In this case you need ***2k+1*** correctly functioning processes, for a total of ***3k+1*** processes
- In both cases we assume that communication failures do not occur

Byzantine Generals Problem



Goal: Reach a common decision to attack at dawn or not

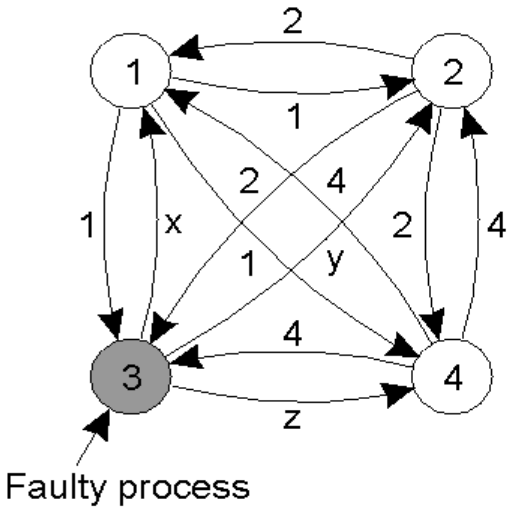
Problems:

1) Messagers could be caught so messages might not get delivered

2) Generals might be traitors and send messages to create confusion

Agreement: Byzantine Generals Problem (1)

How does a process group deal with a faulty member?



(a)

1	Got(1, 2, x, 4)
2	Got(1, 2, y, 4)
3	Got(1, 2, 3, 4)
4	Got(1, 2, z, 4)

(b)

1 Got	2 Got	4 Got
(1, 2, y, 4)	(1, 2, x, 4)	(1, 2, x, 4)
(a, b, c, d)	(e, f, g, h)	(1, 2, y, 4)
(1, 2, z, 4)	(1, 2, z, 4)	(i, j, k, l)

(c)

- The "Byzantine Generals Problem" for 3 loyal generals and 1 traitor.

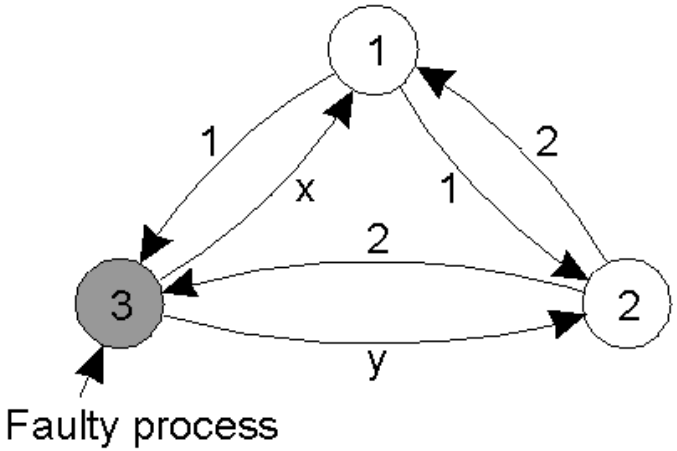
a) The generals announce their troop strengths to the other members of the group by sending a message.

b) The vectors that each general assembles based on (a), each general knows their own strength. They then send their vectors to all the other generals.

c) The vectors that each general receives in step 3. It is clear to all that General 3 is the traitor. In each 'column', the majority value is assumed to be correct.

Agreement: Byzantine Generals Problem (2)

Warning: the algorithm does not always work! ⇒ Two-Army Problem



(a)

1	Got(1, 2, x)
2	Got(1, 2, y)
3	Got(1, 2, 3)

(b)

1 Got	2 Got
(1, 2, y)	(1, 2, x)
(a, b, c)	(d, e, f)

(c)

- The same algorithm as in previous slide, except now with 2 loyal generals and 1 traitor. Note: It is no longer possible to determine the majority value in each column, and the algorithm has failed to produce agreement.
- It has been shown that for the algorithm to work properly, *more* than two-thirds of the processes have to be working correctly. That is: if there are M faulty processes, we need 2M + 1 functioning processes to reach agreement.

Oral Messages

Byzantine Agreement (n nodes, m faults)

- *can be reached using Oral Messages,*
  if ***n >= 3m+1***

Assumptions:

- Every message is delivered correctly
- Receiver knows who sent message
- Absence of message can be detected

Oral Message Algorithm

OM(0)

1. Commander sends his value to every lieutenant
2. Each lieutenant uses the received value or "retreat" if no value received

OM(m)

1. Commander sends his value to every lieutenant
2. Everyone runs OM(m-1) and acts as the commander to send the value received in step 1 to all the other lieutenant
3. Each lieutenant uses the majority value out of the values received

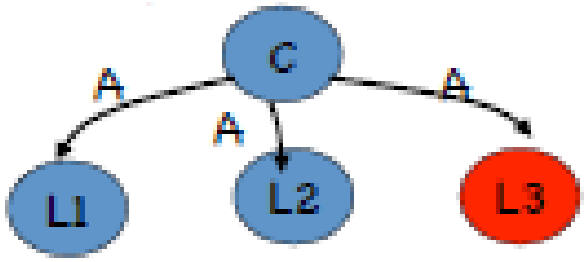
Recursive Algorithm

*(No solution with fewer than **3m+1** generals can cope with **m** traitors)*

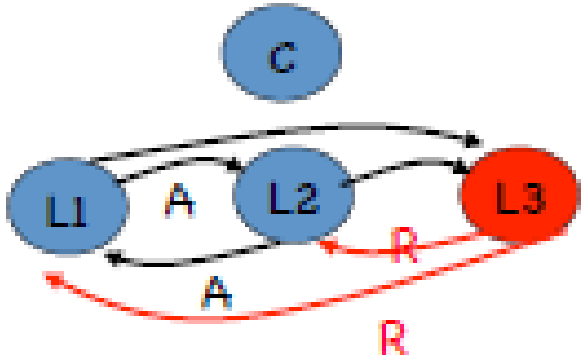
Example: Bad Lieutenant

Scenario: m=1, n=4, traitors=L3

OM(1):



OM(0):???

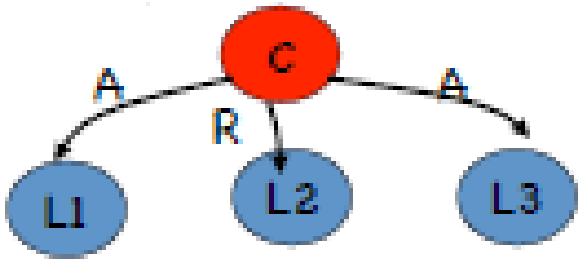


Decision?? L1 = m (A, A, R); L2 = m (A, A, R); Both attack!

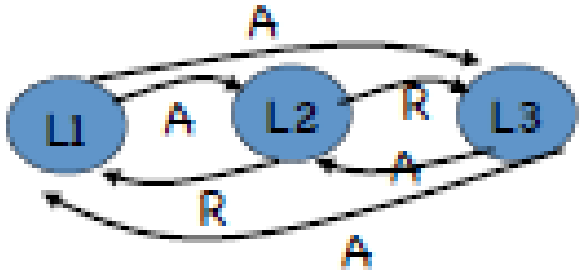
Example: Bad Commander

Scenario: m=1, n=4, traitors=C

OM(1):



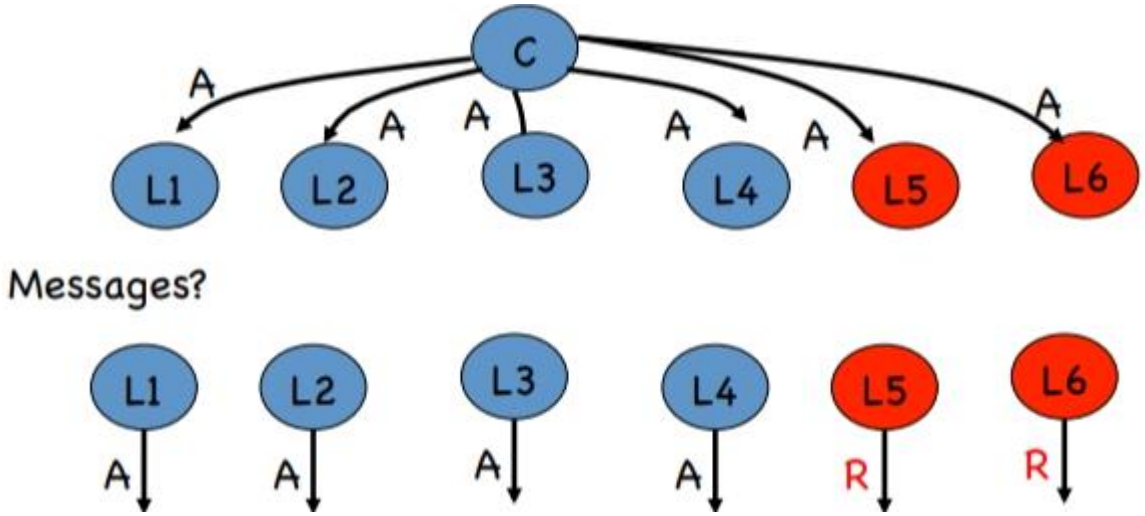
OM(0):???



Decision?? L1=m(A, R, A); L2=m(A, R, A); L3=m(A,R,A); Attack!

Bigger BFT (Bad Lieutenants)

Scenario: m=2, n=7, traitors=L5,L6

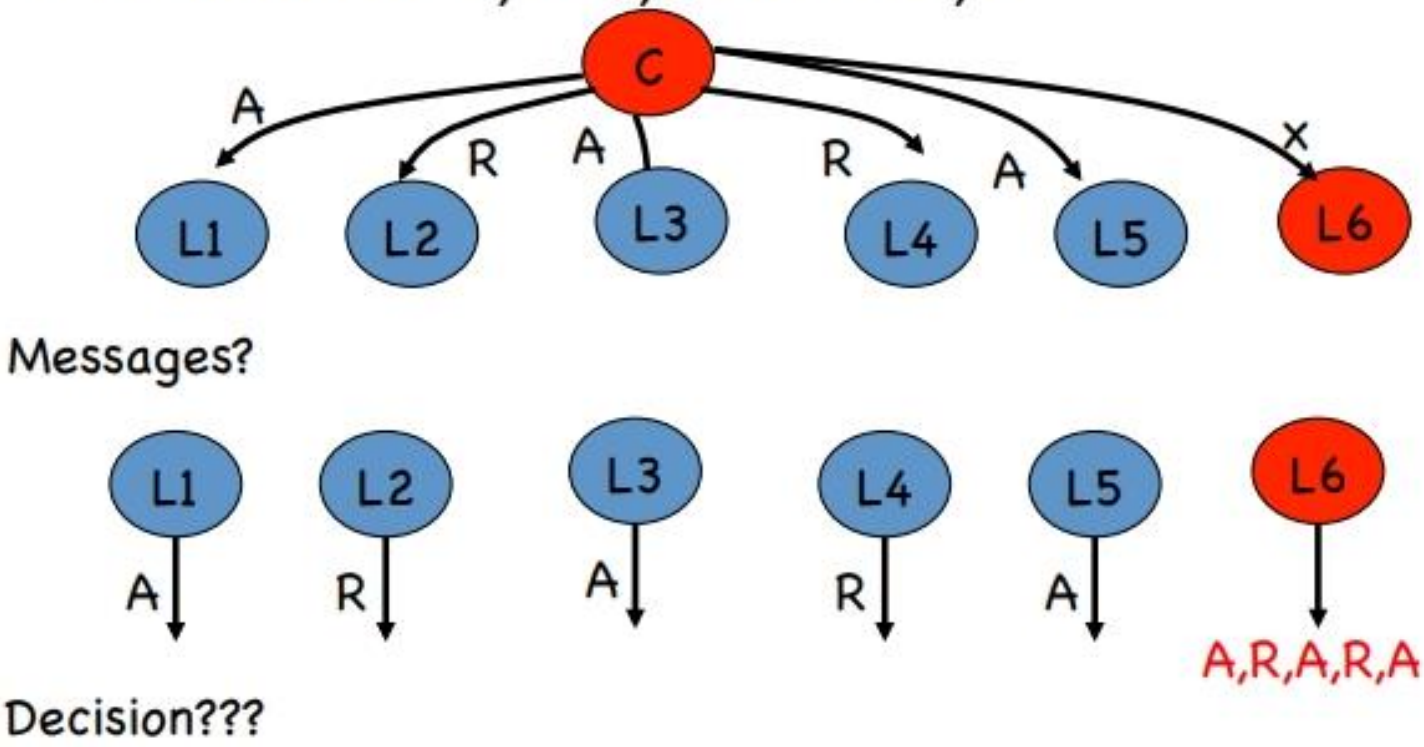


Decision??

https://github.com/AnantJoshiCZ/Byzantine

Bigger BFT (Bad Commander+)

- Scenario: m=2, n=7, traitors=C, L6



Decision with Bad Commander+

- L1: m(A,R,A,R,A,A) ==> Attack
- L2: m(R,R,A,R,A,R) ==> Retreat
- L3: m(A,R,A,R,A,A) ==> Attack
- L4: m(R,R,A,R,A,R) ==> Retreat
- L5: m(A,R,A,R,A,A) ==> Attack
- Problem: All loyal lieutenants do NOT choose same action
  - SO: need more communication – commander + cheater can tip majority

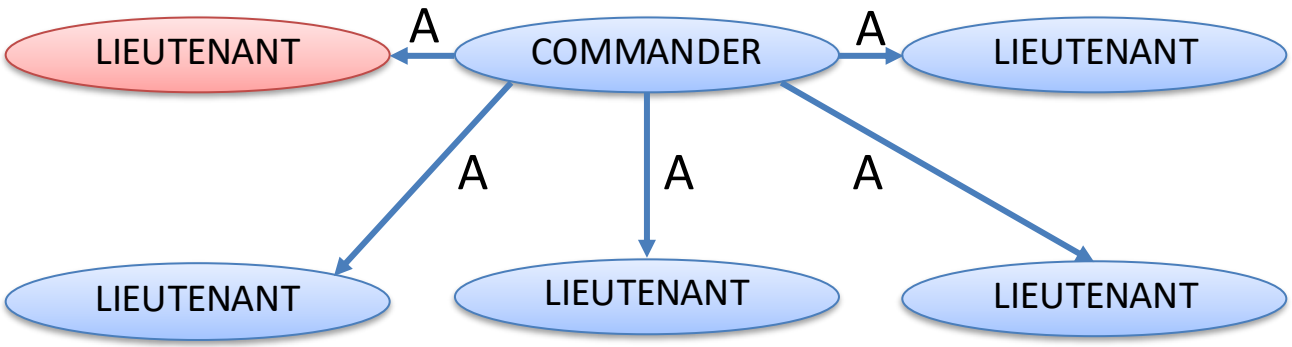
Next Step of Algorithm

- **Key: Verify that lieutenants tell each other the same thing**
  - Requires # rounds = m+1 (0 + m)
  - OM(0): Msg from Lieut i of form: "L0 said v0, L1 said v1, etc..."
    - At each level, get agreement on what the sender sends out, so cannot have some A some R
- What messages does L1 receive in this example?
  - `OM(2): A (commander sends)`
  - `OM(1):  2R, 3A, 4R, 5A, 6A (recv what cmdr sent from others)`
  - `OM(0):  2{  3A, 4R, 5A, 6R} (2 sends all msgs heard in round 1)`
  - `3{2R,  4R, 5A, 6A}`
  - `4{2R, 3A,  5A, 6R}`
  - `5{2R, 3A, 4R,  6A}`
  - `6{ total confusion }`
- All see same messages in OM(0) from L1,2,3,4, and 5
  - Uses majority for each other node – look down columns of matric in OM(0)
- m(1A,2R,3A,4R,5A,-) ==> All attack
- NOTE: # of messages: (n-1)(n-2)(n-3)...(n-m-1) messages for m traitors

Check Yourself

Imagine there are 1 loyal commander, 1 traitor lieutenant, and 4 loyal lieutenants. Please provide a concrete example where Oral Message algorithm is used among these soldiers, yet the loyal lieutenants all are able to follow the commander's command.

You need to briefly describe the process of Oral Message algorithm in your example.

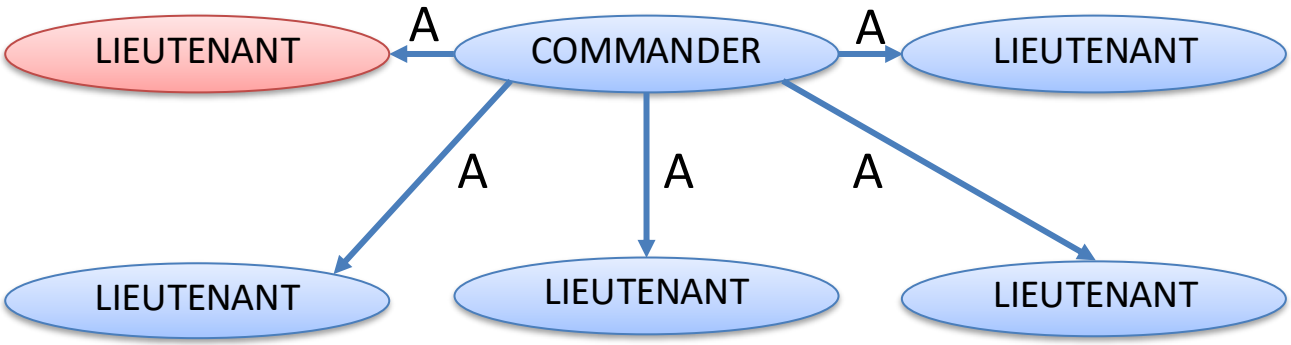


- The red ellipse represents the faulty nodes, while the blue one for the normal nodes

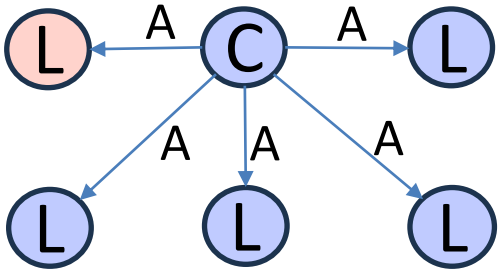
- The commander sends the "Attack" order to all lieutenants to which it connects.

- When each lieutenant act as a commander, a loyal lieutenant sends the original order, "Attack", while the traitor sends "Retreat".

Check Yourself – Solution (1/3)

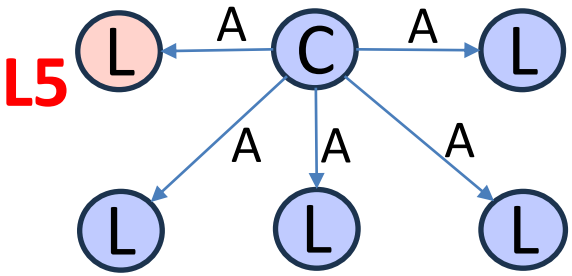


OM(1)

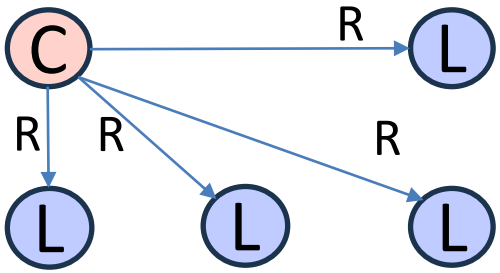


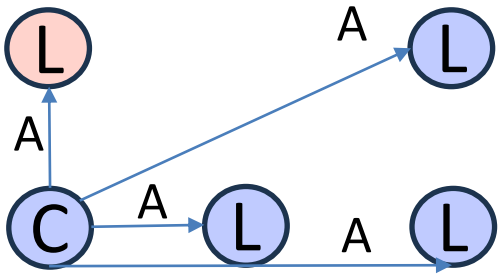
Check Yourself – Solution (2/3)

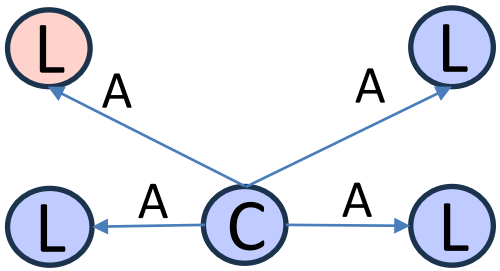
OM(1)

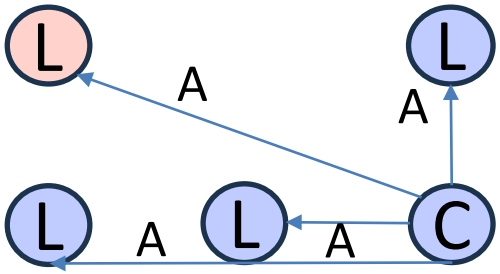


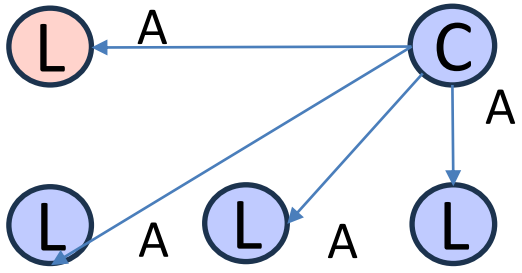
OM(0)











Check Yourself – Solution (3/3)

Lieutenant	Value frm C	Frm L1	Frm L2	Frm L3	Frm L4	Frm L5	Majority Vote
L1	A	-	A	A	A	R	A
L2	A	A	-	A	A	R	A
L3	A	A	A	-	A	R	A
L4	A	A	A	A	-	R	A
L5	x	x	x	x	x	-	x

Number of Messages

Number of Messages

= (n-1)*(n-2)*…*(n-m-1)

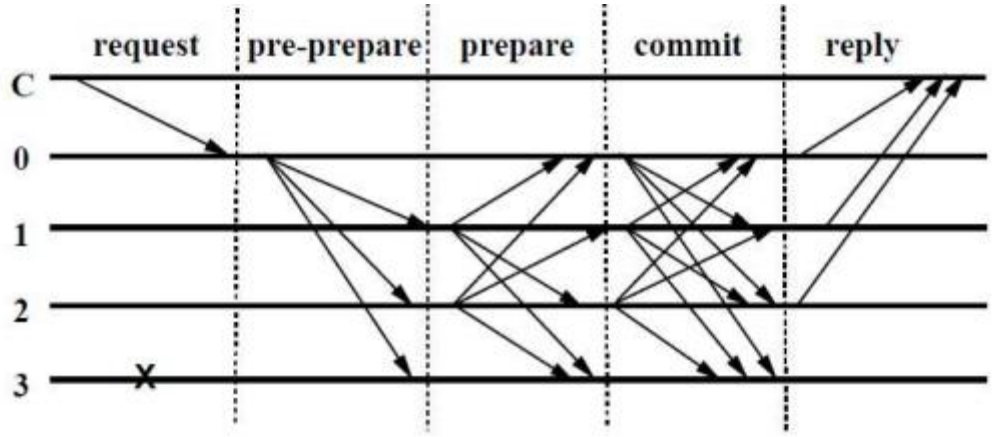
= $O(n^m)$

Exponential in number of Traitors!!



- Practical Byzantine Fault Tolerance
- Signed Message Solution

Practical BFT

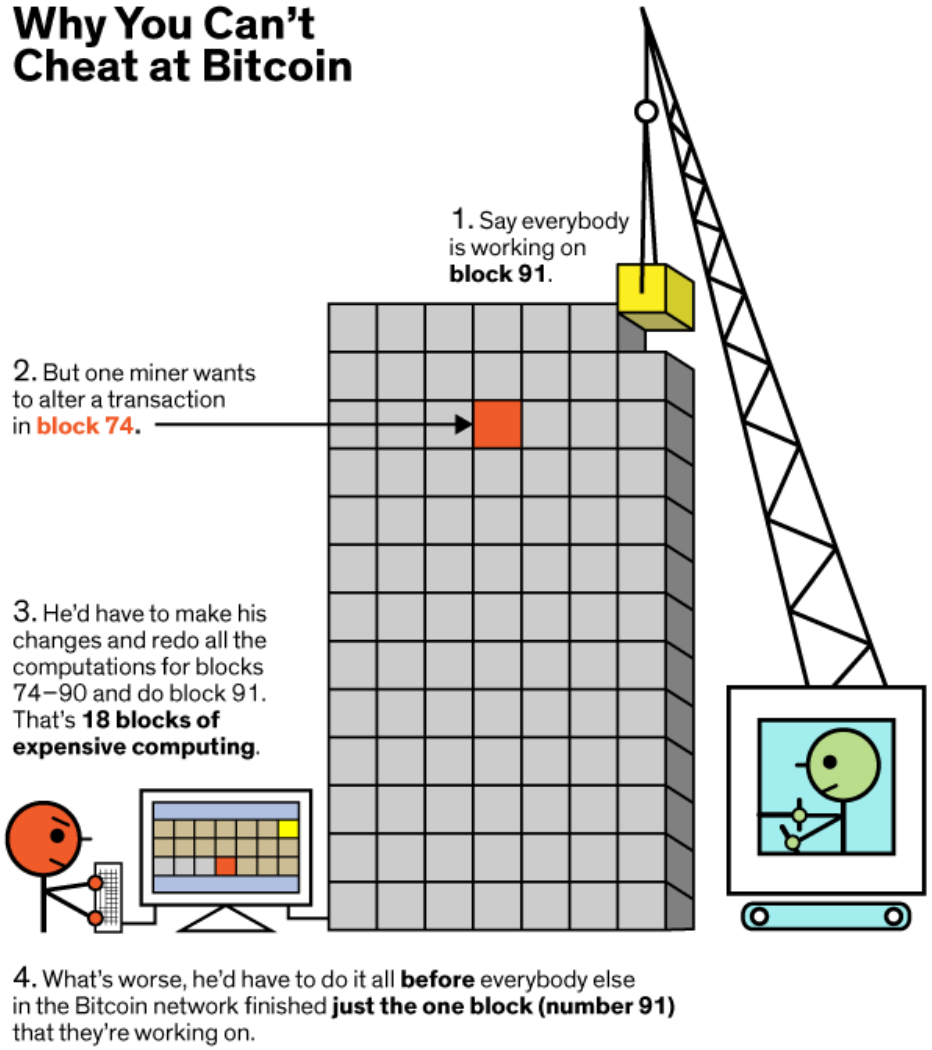


- Client waits for m+1 replies from different replicas with same answer
- If primary is faulty, select new primary
- Many optimizations to reduce overhead

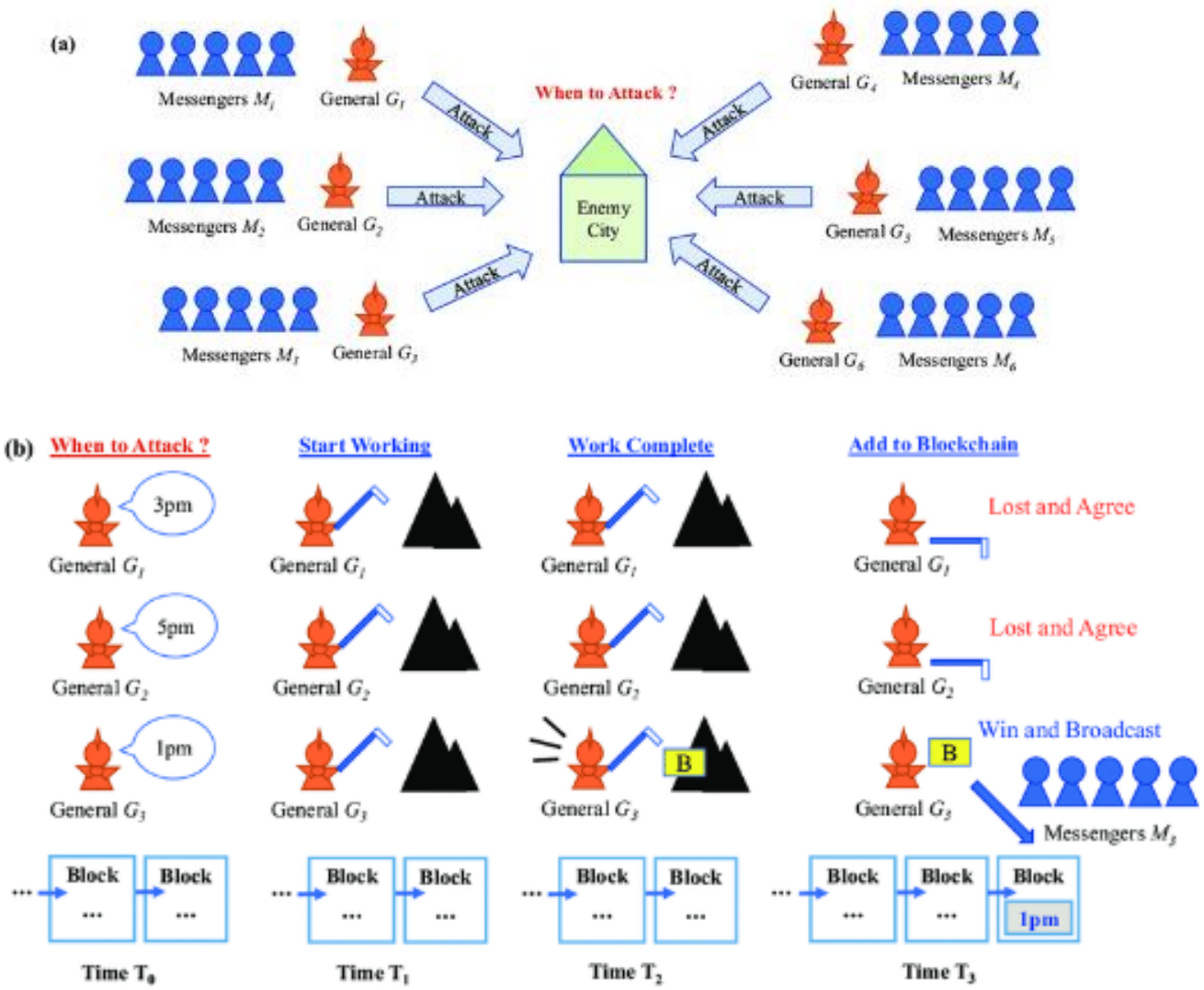
Blockchain

- A solution to the Byzantine Fault Tolerance
  - For any *m*, Algorithm *OM(m)* reaches consensus if there are more than *3m* generals and at most *m* traitors.
- Remove the middleman *while* providing a trusted exchange
- Power of Blockchain
  - Decentralized, self-governing network (no middleman)
  - Trusted network *despite* a lack of central governance
- Proof of Work/ Proof of Stake Algorithm

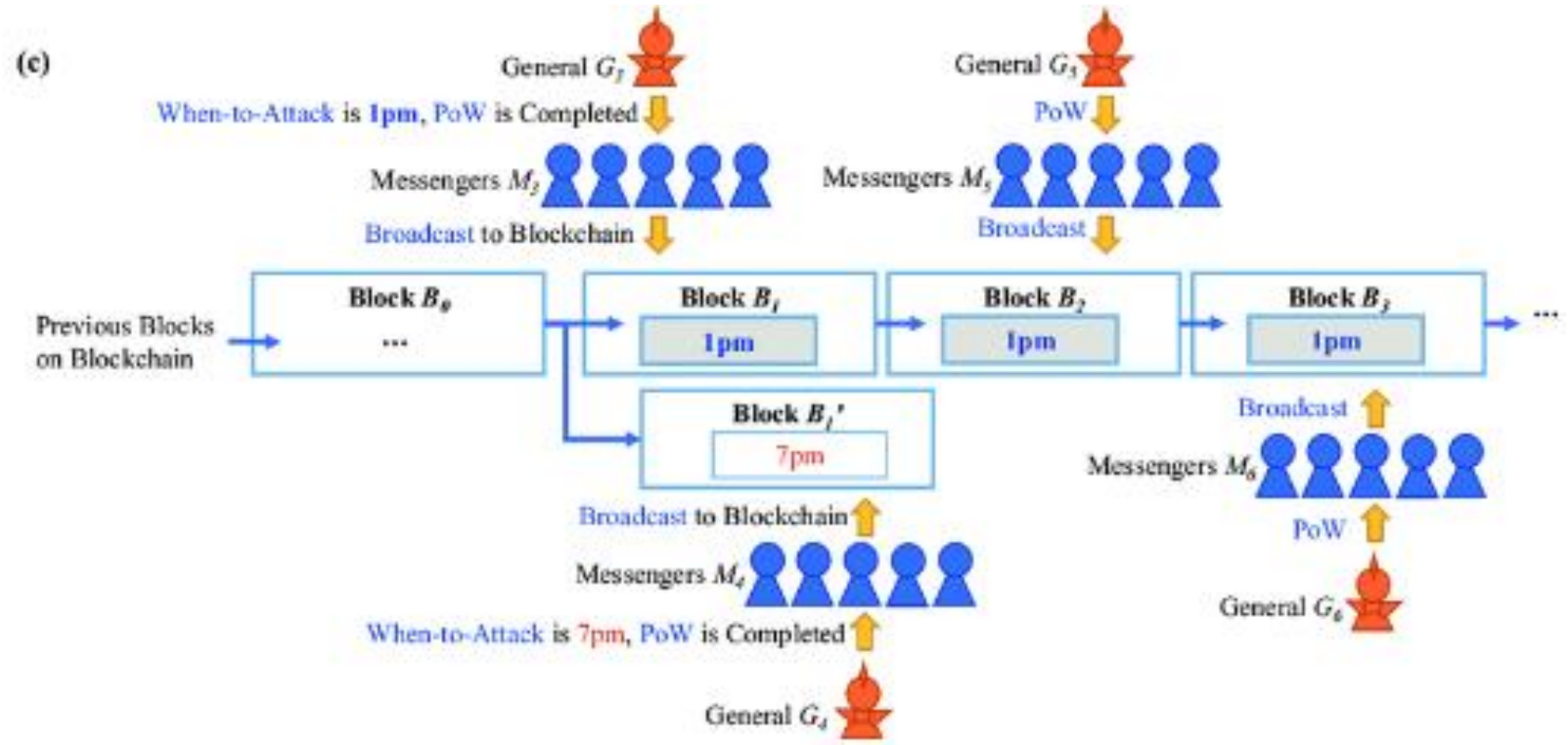
Proof of Work in Bitcoin



BlockChain: Proof of Work



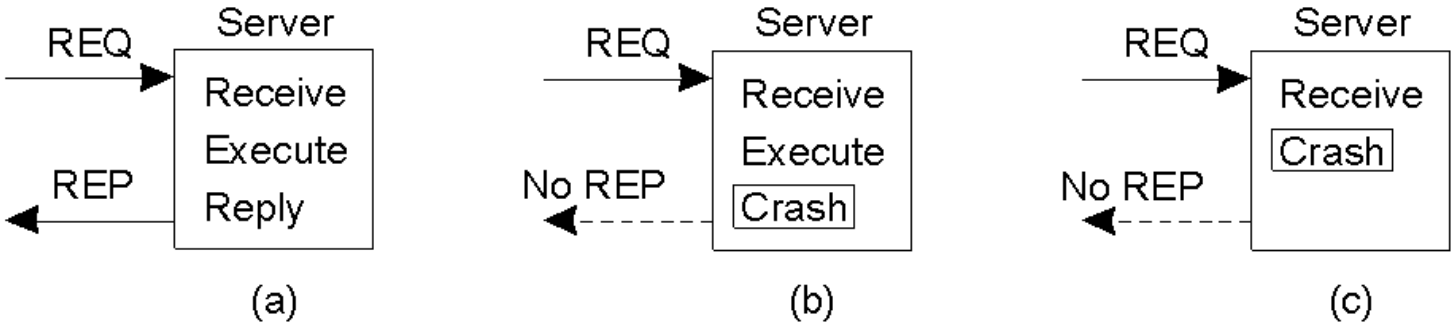
BlockChain: Proof of Work



Reliable Client-Server Communication

- Fault Tolerance in DS considers Communication Failures
  - masking *crash* and *omission* failures
- Reliable Point-to-Point (P2P) communication is established by a reliable transport protocol, such as TCP
  - P2P TCP masks omission failures by guarding against lost messages using ACKs and retransmissions
  - But perform poorly when a crash occurs
- In client/server model, In the case of process failure the following situations need to be dealt with:
  - Client unable to locate server
  - Client request to server is lost
  - Server crash after receiving client request
  - Server reply to client is lost
  - Client crash after sending server request

Lost Request Messages when Server Crashes



A server in client-server communication

a) Normal case
b) Crash after execution
c) Crash before execution

Solutions to Handle Server Failures (1)

- **Client unable to locate server, e.g. server down, or server has changed**

Solution:

- Use an exception handler – but this is not always possible in the programming language used

- **Client request to server is lost**

Solution:

- Use a timeout to await server reply, then re-send – but be careful about idempotent operations (no side effects when re-send)

- If multiple requests appear to get lost assume 'cannot locate server' error

Solutions to Handle Server Failures (2)

- **Server crash after receiving client request**

Problem may be not being able to tell if request was carried out (e.g. client requests print page, server may stop before or after printing, before acknowledgement)

Solutions:

- rebuild server and retry client request (assuming 'at least once' semantics for request)

- give up and report request failure (assuming 'at most once' semantics), what is usually required is exactly once semantics, but this difficult to guarantee

- **Server reply to client is lost**

Client can simply set timer and if no reply in time assume server down, request lost or server crashed during processing request.

Solutions to Handle Client Failures

➢ **Client crash after sending server request** : Server unable to reply to client (orphan request)

Options and Issues:

- **Extermination**: client makes a log of each RPC, and kills orphan after reboot. Expensive.
- **Reincarnation**. Time divided into epochs (large intervals). When client restarts it broadcasts to all, and starts a new time epoch. Servers dealing with client requests from a previous epoch can be terminated. Also unreachable servers (e.g. in different network areas) may later reply, but will refer to obsolete epoch numbers.
- **Gentle reincarnation**, as above but an attempt is made to contact the client owner (e.g. who may be logged out) to take action

Expiration, server times out if client cannot be reached to return reply

Reliable Communication Systems

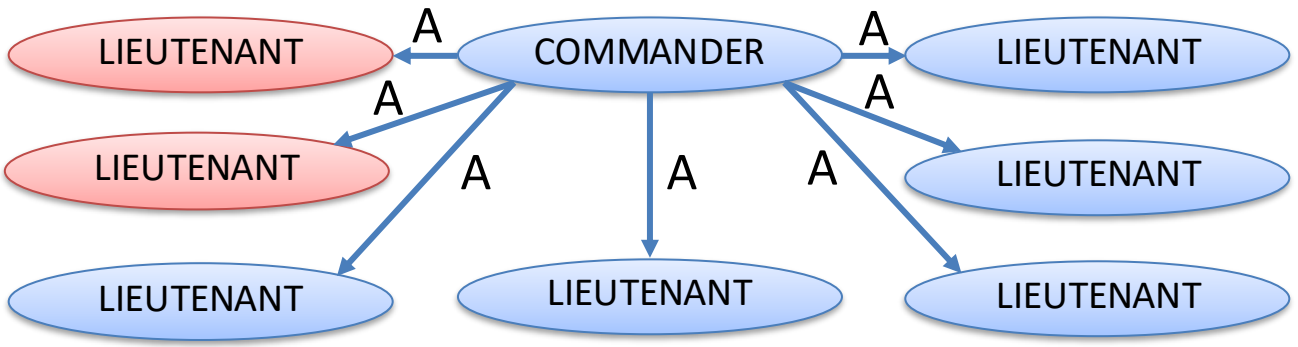
Examine Assumptions:

- Synchronous Communication
- Absence of message can be detected
- Every message that is sent is delivered correctly
- Receiver of a message knows who sent it
- A loyal general's signature cannot be forged, and any alteration of the contents of his signed message can be detected
- Anyone can verify the authenticity of a general's signature

HW03: BFT

Imagine there are 1 loyal commander, 2 traitor lieutenants, and 5 loyal lieutenants. Please provide a concrete example where Oral Message algorithm is used among these soldiers, yet the loyal lieutenants all are able to follow the commander's command.

You need to briefly describe the process of Oral Message algorithm in your example.



- The red ellipse represents the faulty nodes, while the blue one for the normal nodes

- The commander sends the "Attack" order to all lieutenants to which it connects.

- When each lieutenant act as a commander, a loyal lieutenant sends the original order, "Attack", while the traitor sends "Retreat".